

REDES NEURAIS ARTIFICIAIS E MACHINE LEARNING

TEMA 1 – APRENDIZADO PROFUNDO (DEEP LEARNING)

Em termos históricos e práticos, do ponto de vista do estudo das redes neurais, a pesquisa em RNA é caracterizada acontecendo em três ondas desde o seu surgimento (Goodfellow; Bengio; Courville, 2016), como as que se seguem.

1ª onda: refere-se à criação do modelo de neurônio de McCulloch-Pitts em 1943 e a aprendizagem hebbiana de 1949, bem como as implementações do primeiro modelo de RNA, o perceptron de Rosenblatt.

2ª onda: inicia-se com a abordagem conexionista no período de 1980-1985, a partir do trabalho sobre a aprendizagem por retropropagação de Rumelhart.

3ª onda: o início da abordagem *deep learning*, com o trabalho de Hinton *et al.* (2006), o qual demonstrou que o uso da estratégia do **pré-treinamento intensivo de camadas** (*greedy layer-wise pre-training*) pode melhorar a performance das RNA em problemas de alta complexidade. Outros trabalhos importantes para a caracterização desta onda são o de Bengio *et al.* (2007) e o de Ranzato *et al.* (2007), que aplicaram a mesma estratégia para o treinamento de vários outros tipos de RNA.

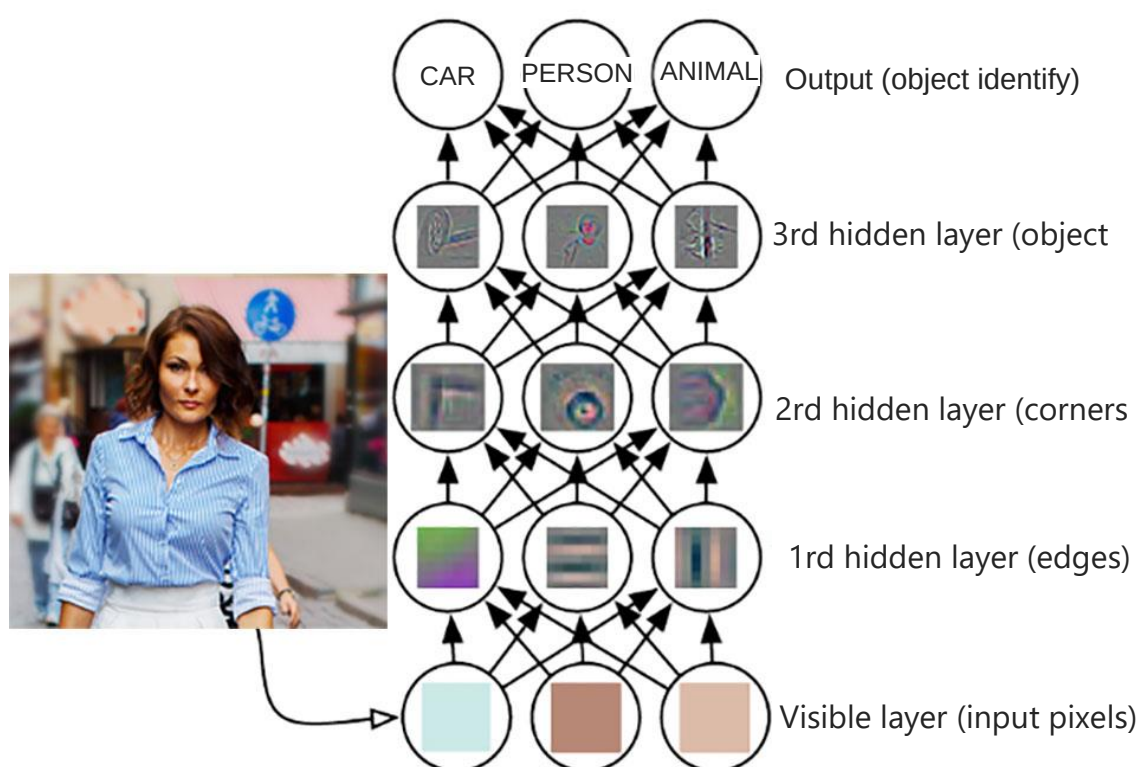
Existem uma série de evidências com base em estudos neurológicos de que o cérebro desenvolve os conceitos de nível superior em estágios, primeiro extraíndo informação de várias camadas de representações úteis e gradualmente mais complexas. Para reconhecer um rosto, em primeiro lugar elementos simples são detectados no córtex visual, como bordas da imagem (mudanças abruptas de intensidade), em seguida, conceitos de nível superior progressivamente, como olhos, boca e características geométricas complexas, independentemente da posição específica na imagem, iluminação, cores etc. (Battiti; Brunato, 2017, p. 104).

O aprendizado profundo resolve o problema central no aprendizado de representação, introduzindo representações que são expressas em termos de outras representações mais simples. O aprendizado profundo permite que o computador construa conceitos complexos a partir de conceitos mais simples. A figura 1 mostra como um sistema de aprendizado profundo pode representar o conceito de imagem de uma pessoa combinando conceitos mais simples, como

cantos e contornos, que por sua vez são definidos em termos de arestas (Goodfellow; Bengio; Courville, 2016, p. 5).

O *deep learning* engloba uma série de técnicas e algoritmos de RNA, aprofundando conceitos de redes neurais alimentadas à frente (*feed forward*), camadas de entrada, ocultas e de saída, função custo, envolvendo o uso de outros tipos de redes tais como redes convolucionais ou convolutivas (*Convolutional Neural Networks-CNN*), redes neurais recorrentes (*Recurrent Neural Networks-RNN*), descida estocástica de gradiente (*Stochastic Gradient Descent*) e máquinas de Boltzmann restritas (*Restricted Boltzmann Machine – RBM*). Tais técnicas estão também aliadas ao uso de hardware contendo unidades de processamento gráfico (*Graphical Processing Units-GPU*) e computação baseada em GPU como tecnologia central para possibilitar o *deep learning* (Ketkar, 2017, p. 1–3; Medeiros, 2018, p. 158).

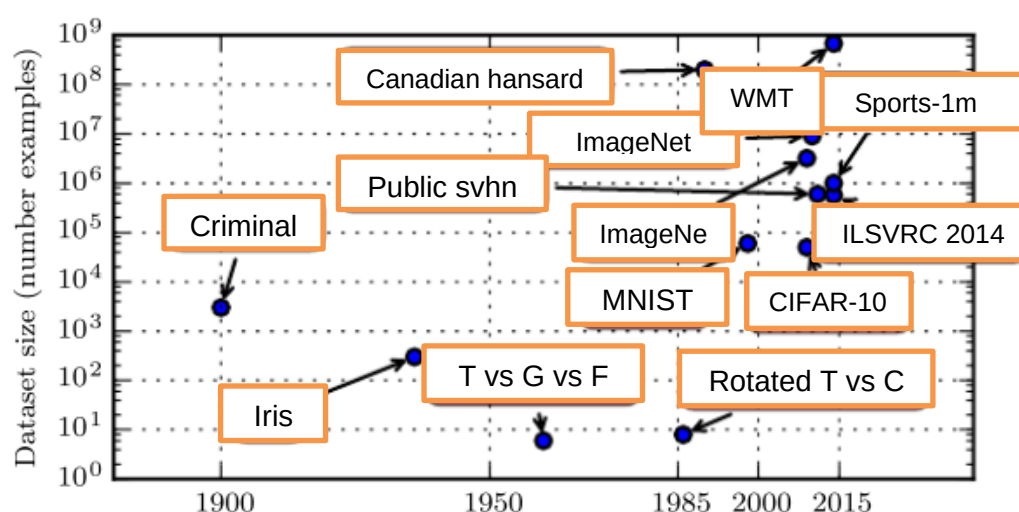
Figura 1 – Representação esquemática de um modelo de rede de aprendizado profundo.



Créditos: FXQuadro/Shutterstock.

Outra razão significativa pela qual as RNA são muito bem-sucedidas hoje, depois de obter comparativamente pouco sucesso no período da 2ª onda, é a existência de **recursos computacionais** para executar modelos de muito maior escala hoje em dia. Da evolução biológica se sabe que os animais se tornam inteligentes quando muitos de seus neurônios trabalham em conjunto. Apenas um neurônio individual ou um pequeno conjunto de neurônios não é particularmente útil (Goodfellow; Bengio; Courville, 2016, p. 20). A figura 2 mostra os conjuntos de dados e sua escala ao longo dos anos, no qual pode-se identificar o conjunto Iris, na faixa de 10^2 instâncias (centenas) e o MNIST, entre 10^4 e 10^5 instâncias (milhares). Alguns *datasets* (WMT) já quase alcançam a faixa de 10^9 , ou seja, quase 1 bilhão de instâncias.

Figura 2 – Escala dos bancos de dados utilizados no treinamento de RNA ao longo dos anos

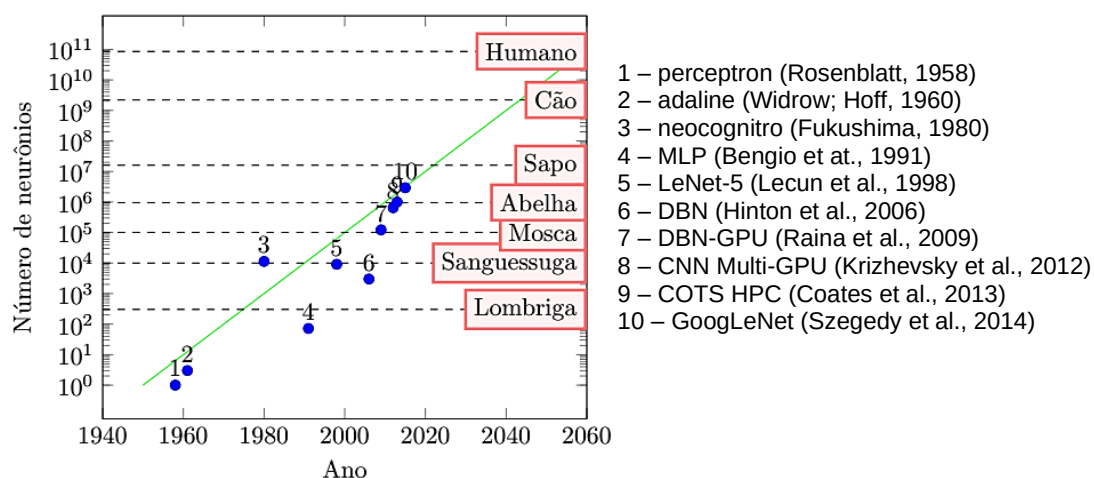


Fonte: Goodfellow *et al.*, 2016.

Deep learning tem melhorado também, ao longo dos últimos anos, a habilidade das RNA no reconhecimento com acurácia ou em tarefas de predição. As RNA construídas nesta perspectiva não precisam que as imagens de entrada sejam recortadas nos objetos os quais se deseja fazer o reconhecimento de padrões. As primeiras RNA, como o perceptron, só podiam reconhecer entre dois tipos de objetos (ou, em alguns casos, a presença ou não de um único tipo de

objeto). As RNA atuais reconhecem normalmente pelo menos 1.000 categorias diferentes de objetos. O maior concurso de reconhecimento de objetos é o Desafio de Reconhecimento Visual em Grande Escala ImageNet (*ImageNet Large Scale Visual Recognition Challenge – ILSVRC*), que é realizado todos os anos. Um momento emblemático no despertar meteórico do *deep learning* aconteceu quando uma rede convolucional (*Convolutional Neural Network - CNN*) venceu esse desafio pela primeira vez e por uma margem muito alta, reduzindo a taxa de erro dos cinco principais modelos de 26,1% para 15,3% (Krizhevsky; Sutskever; Hinton, 2012). Desde então, essas competições são consistentemente ganhas por redes convolucionais profundas e, quando este livro foi escrito, os avanços na aprendizagem profunda reduziram a taxa de erro mais recente dos cinco primeiros nesta competição para 3,6%.

Figura 3 – Evolução da quantidade de neurônios em algumas RNA publicadas, permitindo a comparação com a quantidade de neurônios em alguns seres vivos



Fonte: elaborada com base em Goodfellow *et al.*, 2016.

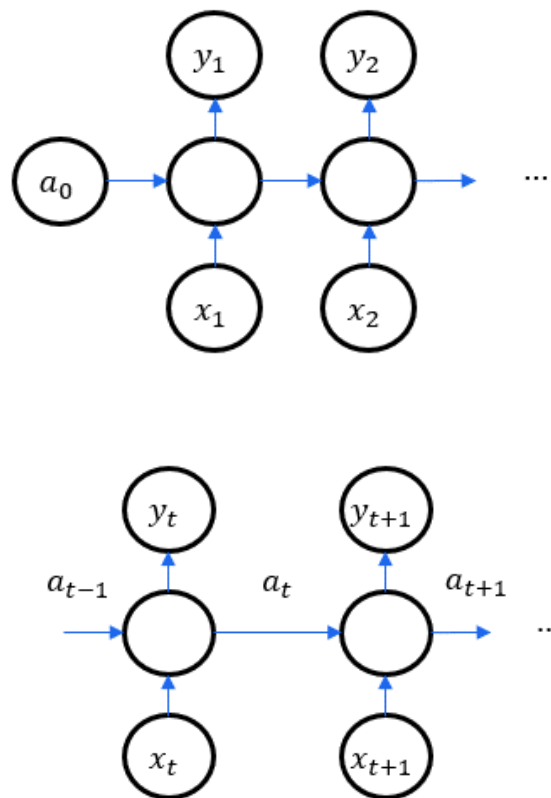
Dentro do contexto do *deep learning*, algumas arquiteturas têm se popularizado quanto ao seu uso na abordagem de problemas complexos com *datasets* massivos:

Redes Convolutivas (*Convolutional Neural Networks - CNN*): são simplesmente redes neurais que usam convolução no lugar da multiplicação geral da matriz em pelo menos uma de suas camadas. O estudo das CNN será aprofundado no próximo tema.

Redes Neurais Recorrentes (*Recurrent Neural Networks – RNN*): são uma família de redes neurais para processamento de dados sequenciais. Assim

como uma rede convolucional é uma rede neural especializada para processar uma grade de valores X , como uma imagem, uma rede neural recorrente é uma rede neural especializada para processar uma sequência de valores $x^1 \dots x^\tau$. Assim como as redes convolucionais podem escalar prontamente para imagens com grande largura e altura, e algumas redes convolucionais podem processar imagens de tamanho variável, as redes recorrentes podem escalar para sequências muito mais longas do que seria prático para redes sem especialização baseada em sequência. A maioria das redes recorrentes também pode processar sequências de comprimento variável (Goodfellow; Bengio; Courville, 2016).

Figura 4 – Representação esquemática de uma rede recorrente, mostrando os elementos ocultos para um processo de ativação específica



A ativação em uma RNN acontece em dois momentos, um para atualizar os elementos internos a_t e outra ativação para atualizar os elementos y_t :

$$a_t = f_1(w_{aa}a_{t-1} + w_{ax}x_t + b_a)$$

e

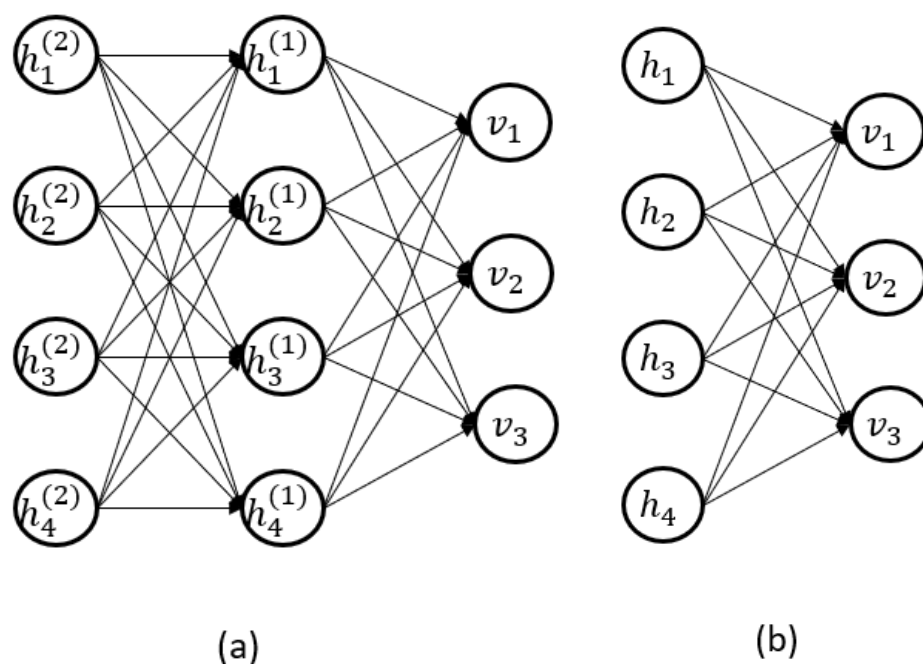
$$y_t = f_2(w_{ay}a_t + b_y)$$

Onde f_1 e f_2 são as funções de ativação, os pesos w_{aa} , w_{ax} e w_{ay} e os bias b_a e b_y são parâmetros compartilhados temporalmente. Na figura 4 está a arquitetura genérica de uma RNN. Os elementos de entrada e saída x e y podem variar em quantidade, dependendo do problema em questão (por exemplo, análise de sentimentos possui várias entradas e uma saída; máquinas de tradução podem ter várias entradas e várias saídas).

Uma integrante da família das RNN é a **LSTM** (do inglês *Long Short-Term Memory*). LSTM refere-se a uma nova arquitetura de rede recorrente contendo um algoritmo de aprendizagem também baseado em descida de gradiente. LSTM foi projetado para superar problemas de atraso de erro. Ele pode aprender a transpor intervalos de tempo em excesso, mesmo no caso de sequências de entrada incompressíveis e barulhentas, sem perda de recursos de intervalo de tempo curto (Hochreiter; Schmidhuber, 1997). LSTM tem sido bem-sucedida em aplicações como reconhecimento de escrita, reconhecimento de fala, geração de escrita, análise de sentimento e tradução de máquina.

Máquinas de Boltzmann Restritas (*Restricted Boltzmann Machine – RBM*): refere-se a uma rede cuja arquitetura contém neurônios estocásticos. As máquinas de Boltzmann foram originalmente introduzidas como uma abordagem conexionista genérica para aprender distribuições de probabilidade arbitrárias sobre vetores binários. A máquina de Boltzmann se torna mais potente quando nem todas as variáveis são observadas. Nesse caso, as variáveis latentes podem agir de forma semelhante às unidades ocultas em um perceptron multicamadas e modelar interações de ordem superior entre as unidades visíveis. As **máquinas de Boltzmann restritas** são alguns dos blocos de construção mais comuns de modelos probabilísticos para *deep learning*. Na figura 5 está ilustrada a representação de uma máquina de Boltzmann. Ela contém neurônios visíveis (v_i) e neurônios ocultos (h_i). Em (a) está uma máquina de Boltzmann com duas camadas ocultas e em (b) uma máquina de Boltzmann restrita, que contém uma camada oculta.

Figura 5 – Representação da arquitetura de uma máquina de Boltzmann com duas camadas ocultas (a) e uma máquina de Boltzmann restrita (b)



Fonte: adaptada de Goodfellow *et al.*, 2016.

TEMA 2 – REDES CONVOLUTIVAS

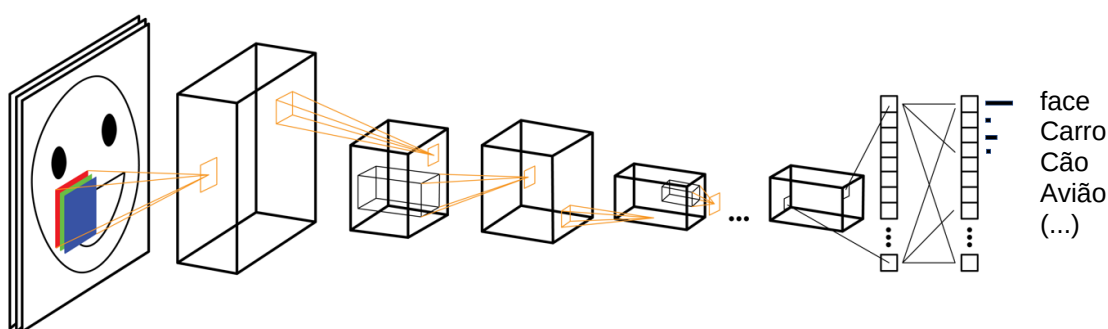
O estudo de RNA sobre a perspectiva do *deep learning* tem nas redes convolutivas um dos grandes desenvolvimentos para o reconhecimento de padrões. A concepção moderna de **redes convolutivas** ou **convolucionais** inicia-se com LeCun *et al.* (1998). Também conhecidas como rede neural convolucional, constituem um tipo especializado de rede neural para processar dados que têm uma topologia conhecida, semelhante a uma grade. Como exemplos, podem ser utilizados dados de séries temporais, que podem ser considerados como uma grade unidimensional coletando amostras em intervalos de tempo regulares; e dados de imagem, que podem ser considerados como uma grade bidimensional de pixels.

As redes convolucionais têm sido extremamente bem-sucedidas em aplicações práticas. O nome “rede neural convolucional” indica que a rede emprega uma operação matemática chamada **convolução**, que é um tipo específico de operação linear. As redes convolucionais são simplesmente redes

neurais que usam convolução no lugar da multiplicação geral da matriz em pelo menos uma de suas camadas (Goodfellow; Bengio; Courville, 2016, p. 330).

Uma rede convolucional é composta de várias camadas que desempenham funções distintas, à medida que vai processando os valores de entrada. Além das camadas que executam a convolução, existem outras que fazem a operação de *pooling* (que poderia ser traduzida como agrupamento ou redução de resolução), camadas com função de ativação específica como a **retificação linear** e camadas que fazem a poda (*dropout*). A camada final geralmente é uma camada totalmente conectada, a qual deve ser treinada de maneira similar ao MLP para fornecer os resultados (figura 6).

Figura 6 – Diagrama esquemático de uma rede convolucional e suas principais camadas



2.1 Convolução

A convolução é uma operação muito aplicada em processamento e filtragem de sinais e imagens. A definição matemática de convolução é uma operação linear que atua sobre duas funções dadas, gerando uma terceira que é a soma do produto que se desloca ao longo de uma região entendida como a superposição entre elas, em um domínio contínuo. A convolução utilizada em CNN opera sobre domínios discretos de valores. Em uma dimensão, a convolução entre as funções f e g , simbolizada como $f * g$, pode ser calculada como:

$$(f * g)(k) = \sum_{i=-\infty}^{\infty} f(i)g(k - i)$$

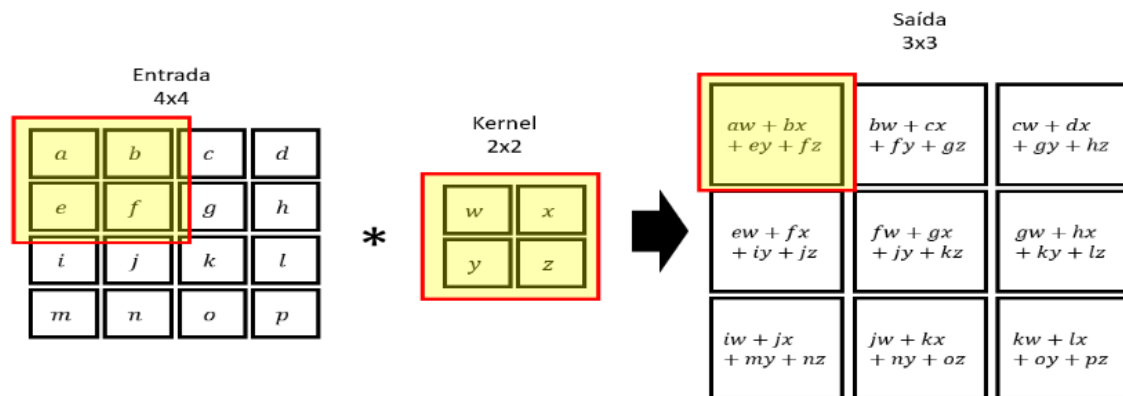
Para o caso de uma imagem em duas dimensões I , a convolução utiliza um **kernel**, simbolizada a seguir como K (também chamado de **filtro**), que

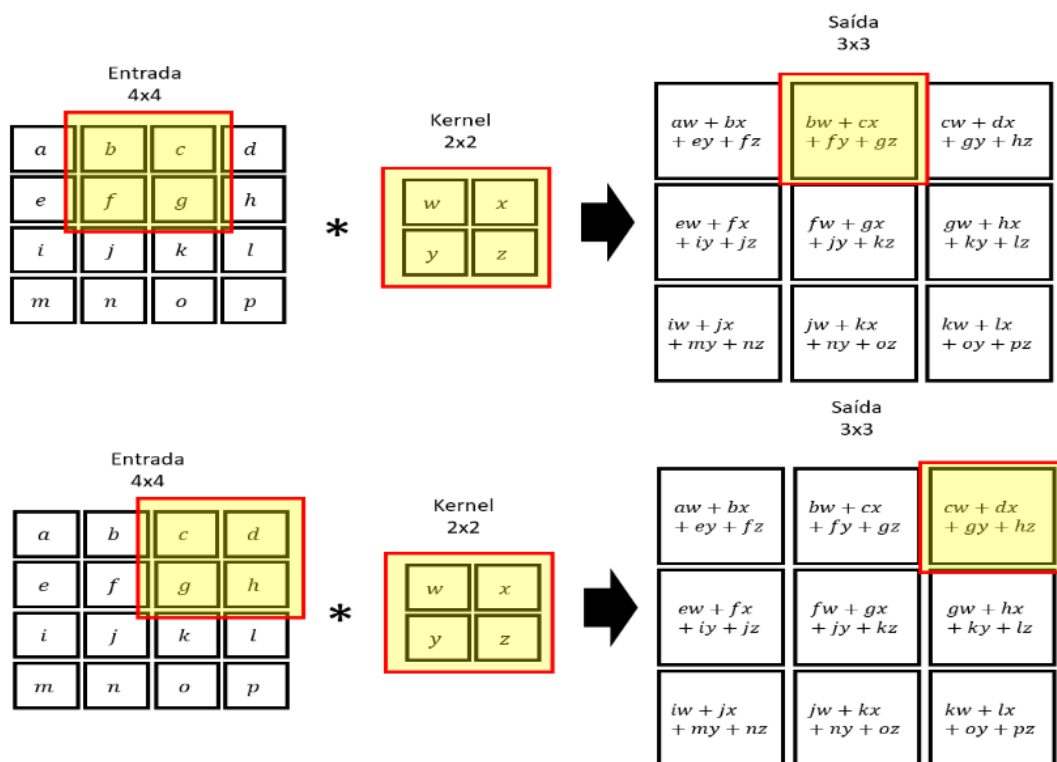
também pode ser interpretada como uma imagem de tamanho bem menor, a convolução gera outra imagem S que é calculada como:

$$S(i, j) = (K * I)(i, j) = \sum_m \sum_n I(m, n) K(i - m, j - n)$$

Apesar da extensão da fórmula, seu entendimento é bastante intuitivo. O resultado da convolução é visualizado como se fosse o deslocamento do kernel sobre a imagem de entrada, produzindo uma imagem de saída.

Figura 7 – Representação da operação de convolução discreta sobre uma imagem em duas dimensões

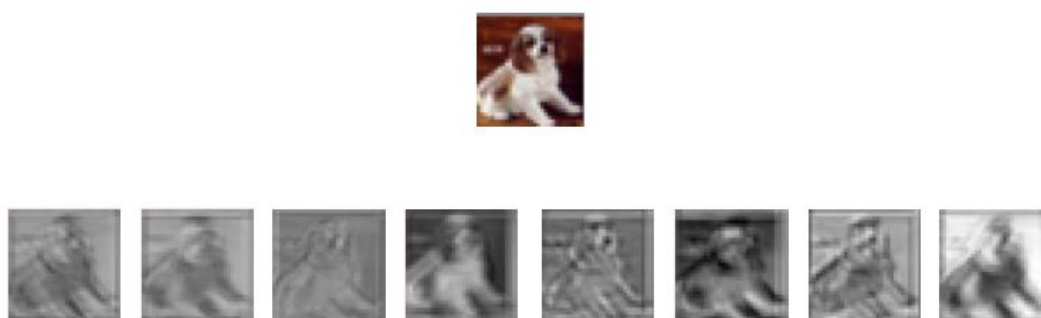




Na Figura 7, o kernel, que é uma região 2x2, se desloca sobre a imagem de entrada 4x4, gerando uma imagem de saída de tamanho menor, de dimensões 3x3. Os valores das células (a, b, e, f) e do kernel (w, x, y, z) com o mesmo índice são multiplicados entre si e somados ($aw + bx + ey + fz$), para gerar o valor da primeira célula de saída. O kernel é, então, deslocado uma célula para a direita, e o processo de multiplicação agora se dá com as quatro células (b, c, f, g), gerando a saída ($bw + cx + fy + gz$). Após terminar a linha, o processo recomeça na próxima, até fazer a varredura em toda a imagem de entrada para gerar cada célula da imagem de saída.

As dimensões das saídas dependerão da dimensão do kernel utilizado (caso seja utilizado um kernel 1x1, por exemplo, a saída terá o mesmo tamanho da entrada). A presença de uma camada convolutiva pode ser feita com um banco de kernels, gerando diversas saídas. No caso do reconhecimento de caracteres MNIST, por exemplo, uma imagem de tamanho 28x28 pixels passando por 9 kernels diferentes irá gerar um volume 28x28x9, aumentando, dessa forma, as possibilidades da extração de características. Na figura 8 está ilustrado um exemplo com imagem do dataset CIFAR-10 com a visualização de oito imagens filtradas por uma camada de convolução.

Figura 8 – Saída de uma camada de convolução com um banco de oito filtros, a partir da imagem superior, da rede ConvNetJS do dataset CIFAR-10. A imagem 32x32x3 produz um volume 32x32x16 (são mostrados apenas oito na figura), com um kernel de convolução 5x5x3



Fonte: Stanford University, [S.d.].

2.2 Retificação Linear (ReLU)

Nas redes convolucionais, após a camada de convolução propriamente dita, é colocada uma camada de **retificação linear**. Em uma camada de retificação, existem unidades denominadas de retificadores lineares (também denominados de ReLU – *Rectifier Linear Unit*). Esta retificação consiste em uma função de ativação aplicada sobre a saída da operação de convolução (Goodfellow; Bengio; Courville, 2016, p. 174). A retificação linear é executada mediante a expressão $f(x) = \max(0, x)$. Ou seja, os valores negativos que porventura sejam obtidos no cálculo da convolução são zerados, ficando apenas os valores acima de zero e positivos. Outra função que pode ser utilizada no lugar da ReLU é a **leaky ReLU** (retificador linear com “vazamento”). Ao invés de adotar zero como limite inferior, adota-se um valor bem pequeno. A figura 9 mostra o comportamento da função de ativação ReLU.

A camada ReLU deixa o tamanho do volume após a convolução inalterado. O propósito da camada ReLU é filtrar a informação que se propaga à frente na rede convolucional. O uso de um retificador linear tem vantagens sobre uma função de ativação como a sigmoide, por exemplo, como i) facilitar o cálculo dos valores (gradientes) para o treinamento das camadas de convolução; ii) performance, pois a fórmula é muito simples; e iii) representações esparsas, que

permitem uma melhor distinção das características extraídas. Na figura 10 pode ser notado o efeito da ativação com ReLU sobre uma camada de convolução.

Figura 9 – Função de ativação de retificação linear (ReLU)

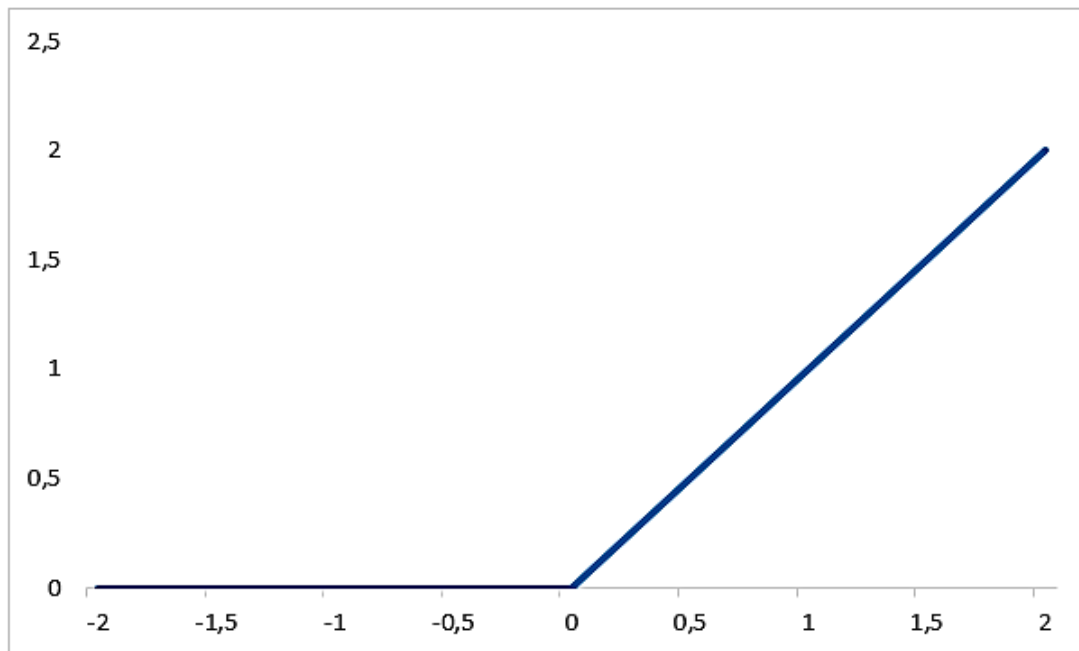
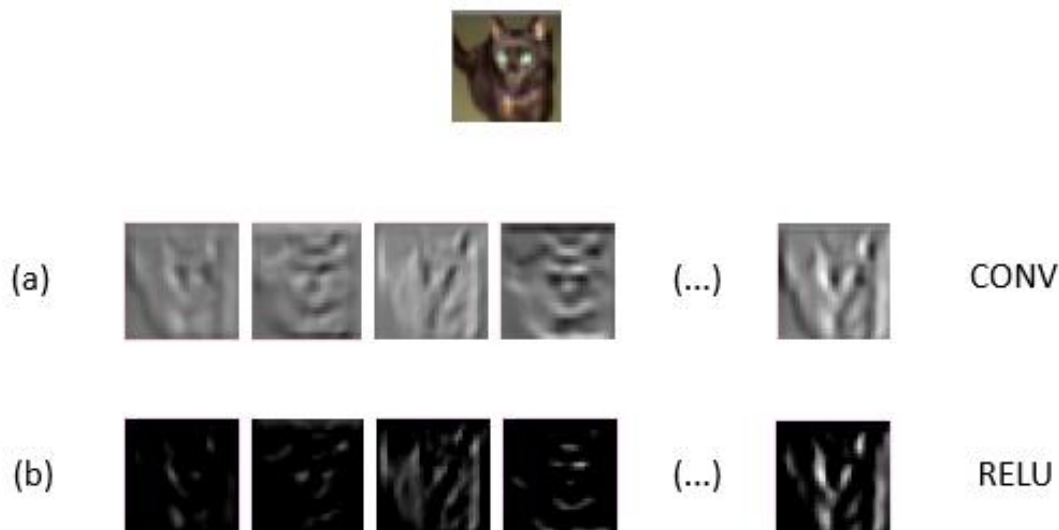


Figura 10 – Comparação entre uma camada de convolução e uma camada com retificador linear, a partir da imagem superior, de um exemplo da rede ConvNetJS do dataset CIFAR-10. Em (a), a imagem 32x32 produz um volume em na camada de convolução (CONV, 32x32x16); em (b), outro volume na camada de retificação linear (RELU, 32x32x16). Pode-se verificar na camada RELU a predominância das características extraídas das camadas de convolução



Fonte: Stanford University, [S.d.].

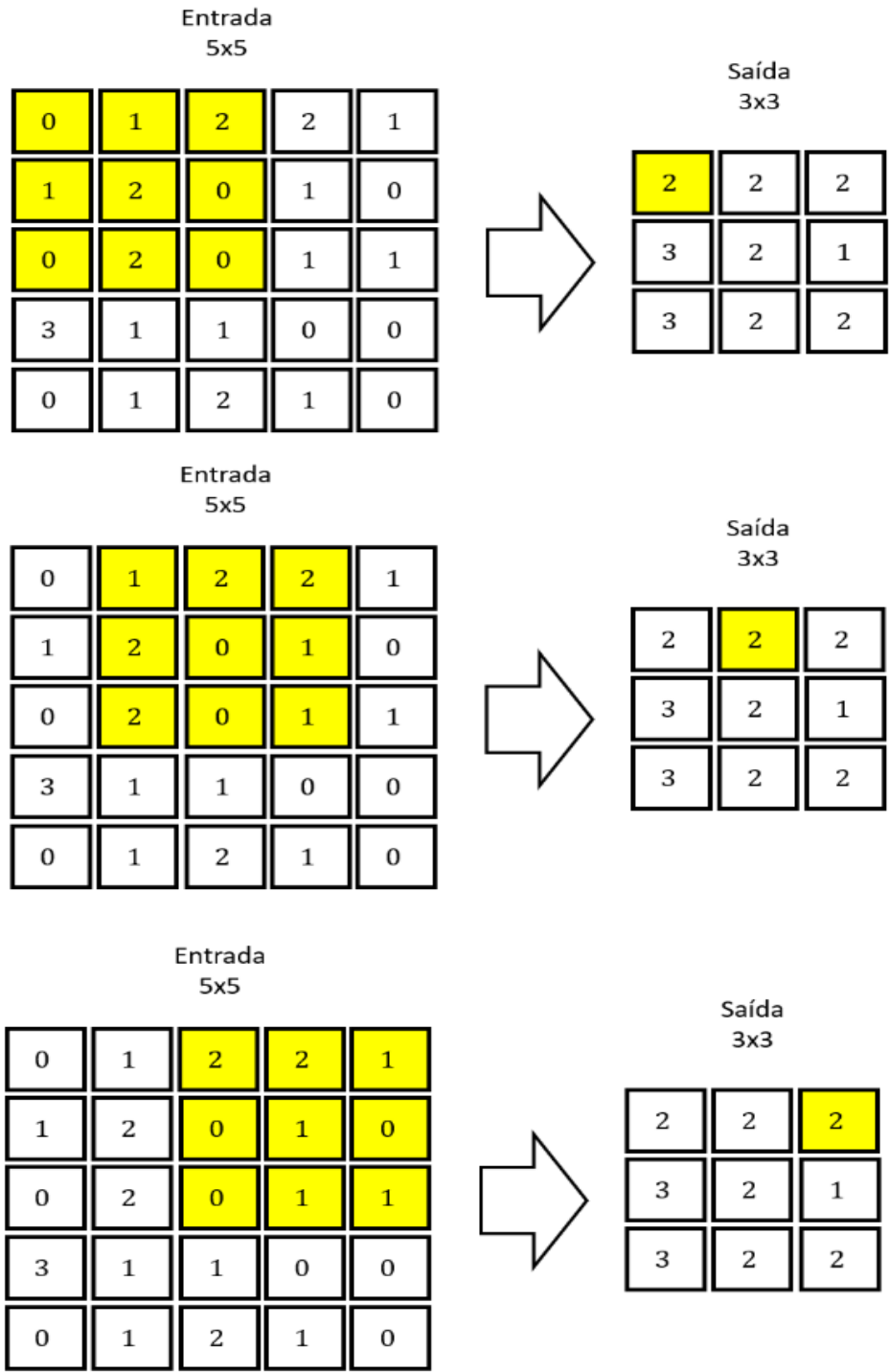
2.3 Pooling

Após a camada de convolução e de retificação linear, a próxima operação essencial em redes convolutivas é a **pooling**. Uma camada de *pooling* produz na sua saída uma estatística dos vizinhos próximos que são considerados como entrada. Por exemplo, caso se considere uma camada de *pooling* com uma entrada 5x5 produzindo uma saída 3x3, cada pixel na saída irá corresponder a uma janela 3x3 que se desloca sobre a entrada 5x5 (figura 11). No caso de a operação resultar no valor máximo da janela, a operação se denomina de **max pooling**. Ou seja, para cada janela há um ponto vencedor que será o ponto de saída. Outras operações de *pooling* podem ser efetuadas, como o uso de média simples ou ponderada (Goodfellow; Bengio; Courville, 2016, p. 339).

Entende-se que a tarefa de uma camada de *pooling* é simplificar a informação para a saída em uma CNN (Nielsen, 2018, p. 174). Uma camada de *pooling* opera como uma subamostragem em uma rede convolutiva. Na medida em que uma CNN tenha vários conjuntos convolução-retificador linear-*pooling*, ela vai reduzindo gradativamente o tamanho da imagem original, até chegar nas camadas com neurônios totalmente conectados (*Fully Connected – FC*) que procedem com a classificação no estilo MLP. Na figura 12 é ilustrado este processo com as camadas combinadas, mostrando que a partir de uma imagem original de tamanho 32x32, produz-se uma camada de convolução (CONV)

também 32x32, a camada de retificação linear (RELU) de 32x32 e a camada de pooling (POOL) 16x16. É comum nas arquiteturas com redes convolucionais utilizar várias camadas combinadas CONV-RELU-POOL.

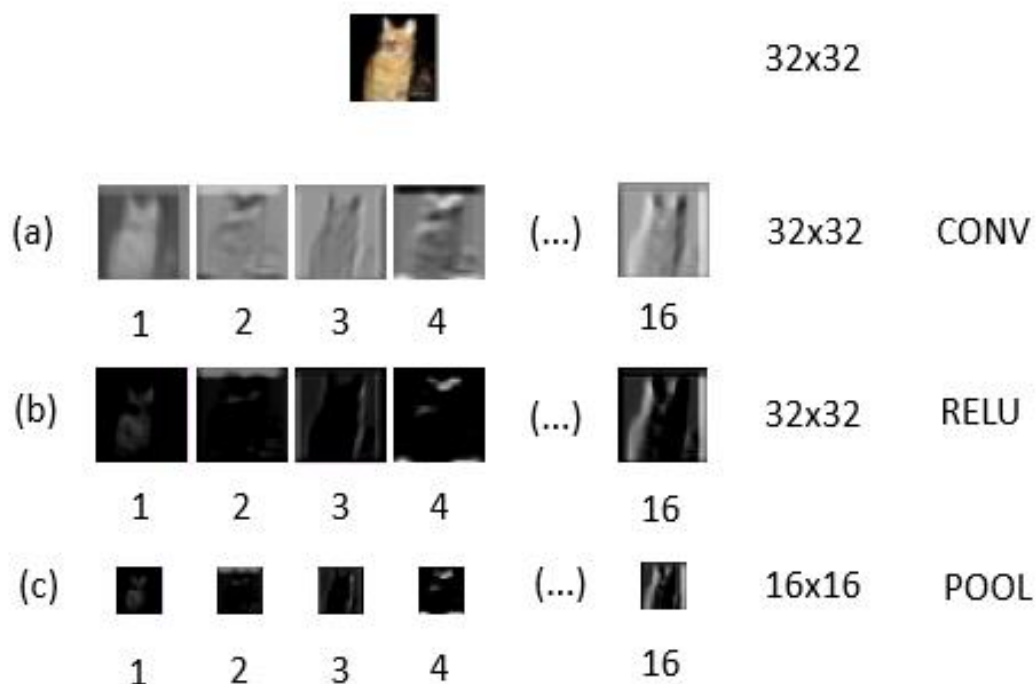
Figura 11 – Camada de *pooling* para uma entrada 5x5 e saída 3x3, utilizando o valor máximo como critério (*max pooling*)



2.4 Achatamento (*Flatten*)

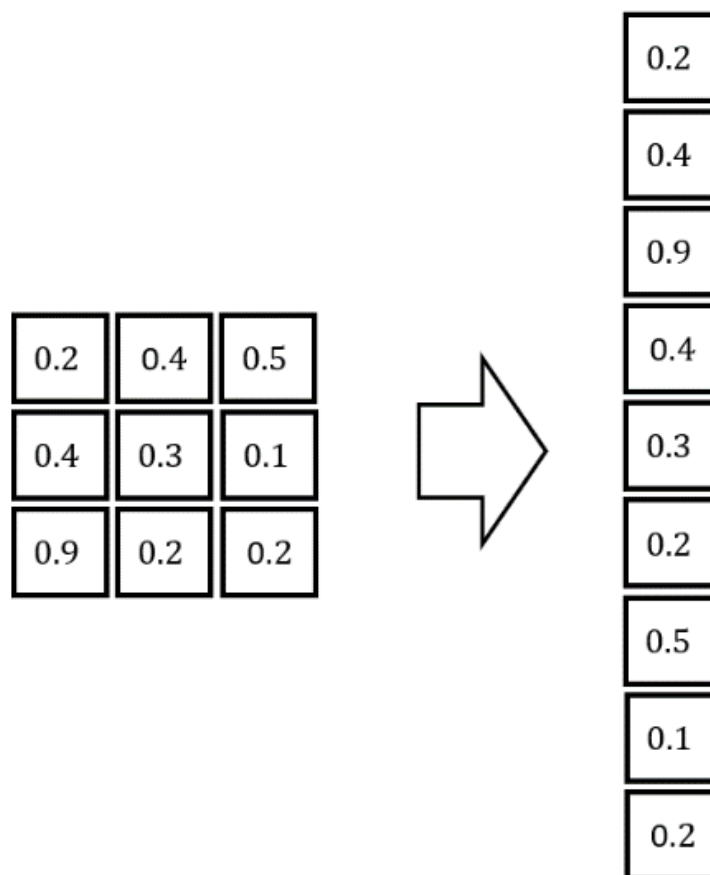
A camada de achatamento em uma CNN faz, na verdade, o alinhamento das camadas bidimensionais em uma camada com uma dimensão apenas. Ela não faz nenhum processamento, sendo útil apenas para o alinhamento para as camadas totalmente conectadas que irão executar a classificação. Este processo pode ser visualizado na figura 13.

Figura 12 – Saída de uma sequência convolução-retificador linear-*pooling*, a partir da imagem superior, da rede ConvNetJS do dataset CIFAR-10. Em (a), a imagem 32x32 produz um volume em na camada de convolução (CONV, 32x32x16); em (b), outro volume na camada de retificação linear (RELU, 32x32x16); e em (c), a camada de *pooling* (POOL, 16x16x16), que utiliza um kernel 2x2



Fonte: Stanford University, [S.d.].

Figura 13 – Operação de achatamento (*flatten*), preparando o vetor para a rede totalmente conectada



2.5 Rede Totalmente Conectada (*Fully Connected*)

A rede totalmente conectada, como camada final de uma rede convolucional, implementa o processo de classificação. Isto é implementado por um perceptron multicamada (MLP), que utiliza como entrada o vetor após a camada de *flatten*. O treinamento atualiza os pesos sinápticos que conectam as unidades entre si. A camada de saída permite que se utilize os neurônios para indicar as classes das amostras de treinamento ou de teste. É comum o uso do acrônimo FC para indicar a última camada da rede convolucional. As camadas com neurônios totalmente conectados também são denominadas de **camadas densas** (*dense layers*).

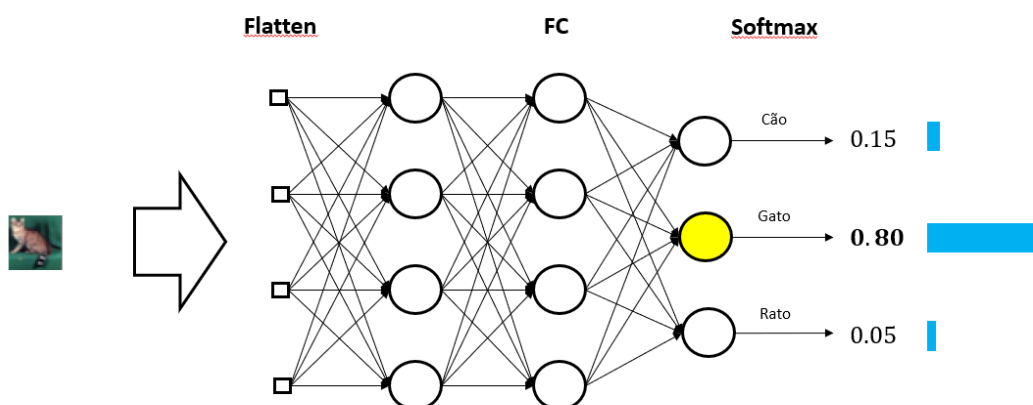
Para o processo de classificação, utiliza-se uma função **softmax**. Esta função faz com que a saída do neurônio seja normalizada (Goodfellow; Bengio; Courville, 2016, p. 81). Durante a apresentação de uma amostra à rede convolucional no treinamento, associa-se o maior valor ao neurônio que corresponde à classe. Na fase de teste, a rede convolucional irá apresentar na saída diferentes valores para os neurônios e espera-se que o neurônio com

maior valor seja aquele da classe ao qual a amostra pertence. A função softmax é calculada com a expressão:

$$f_j(z) = \frac{e^{z_j}}{\sum_{k=1}^N e^{z_k}}$$

A função softmax calcula o valor do neurônio a partir da função exponencial, dividindo pela soma dos valores de todos os neurônios de saída também pela função exponencial. Esse cálculo garante que o valor de saída fique na faixa [0,1]. Para a classe correta, espera-se que o valor esteja mais próximo de 1 do neurônio de saída, e todos os outros estejam próximos de zero. Na figura 14 está ilustrado um exemplo da função softmax com três classes distintas. A normalização garante que a soma das probabilidades das três classes seja igual a 1.

Figura 14 – Ilustração do uso da função softmax, indicando a classe esperada após a apresentação da imagem à rede convolucional



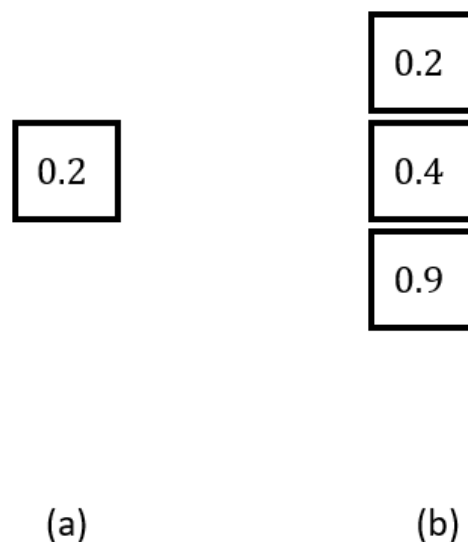
O treinamento da rede também faz com que o sinal do erro se propague, além da camada totalmente conectada, às camadas anteriores de convolução, retificação linear e *pooling*. Desta forma, todos os pesos da rede convolucional são ajustados para que a rede apresente a melhor performance de classificação. Outras camadas podem ser adicionadas, dependendo da arquitetura a ser explorada, como a camada de poda (*dropout*), que irá retirar os pesos que contribuirão muito pouco para a ativação, fazendo com que a performance geral da rede seja aumentada.

TEMA 3 – BIBLIOTECAS E APIS PARA *DEEP LEARNING* (TENSORFLOW E KERAS)

Uma das bibliotecas utilizadas na linguagem Python para o desenvolvimento de modelos em *deep learning* é a Tensorflow. Tensorflow¹ é uma plataforma *open source* própria para o desenvolvimento de modelos de machine learning. Foi desenvolvida pelo Google Brain Team e constitui numa API de baixo e alto nível para diversas operações (adição, multiplicação e diferenciação) e ainda fornece diversos modelos de ML.

Tensorflow foi construída para permitir o uso da estrutura matemática denominada de **tensor**. Um tensor é uma estrutura que generaliza vetores ou matrizes para dimensões mais altas. Dessa forma, em um tensor, existe um conjunto de números organizados em um formato específico. Exemplos de tensores podem ser visualizados na figura 15.

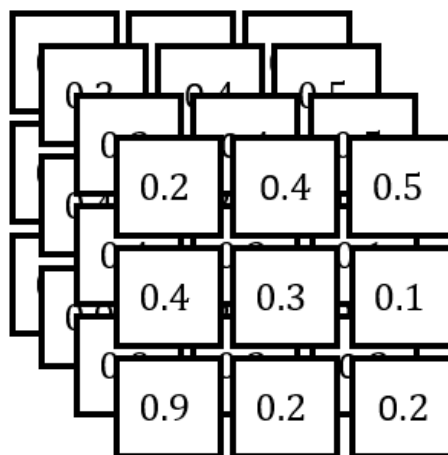
Figura 15 – Exemplos de tensores de várias dimensões: Em (a), um tensor de dimensão zero; em (b), um tensor de dimensão 1; em (c) um tensor de dimensão 2; e em (d), um tensor de dimensão 3.



¹ Disponível em: <<https://www.tensorflow.org/>>. Acesso em: 4 out. 2021.

0.2	0.4	0.5
0.4	0.3	0.1
0.9	0.2	0.2

(c)



(d)

A generalização na biblioteca é importante para que o Tensorflow consiga lidar com estruturas de dados de diferentes dimensões que são requeridas para o *deep learning*. Tensorflow também permite a integração com outras estruturas de bibliotecas como o NumPy, por exemplo, sendo bastante similar às funções e estruturas que esta biblioteca oferece.

Keras² é uma biblioteca de alto nível baseada em Tensorflow que permite a criação de modelos em machine learning e deep learning com bastante flexibilidade. Keras também utiliza a biblioteca Theano³ para otimizar e avaliar expressões matemáticas que envolvam matrizes multidimensionais (tensores) de forma eficiente. Outras bibliotecas recomendáveis para utilizar juntamente com Keras são NumPy, Pandas, Scikit-learn, Matplotlib, Scipy e Seaborn (Keras, 2019).

Assim, Keras tira proveito de várias técnicas de otimização para fazer API de rede neural de alto nível mais fácil e com mais desempenho, suportando os seguintes recursos:

- API consistente, simples e extensível;
- estrutura mínima, obtenção de resultados de forma rápida;
- suporta múltiplas plataformas e backends;
- executa tanto em hardware com CPU e GPU; e

² Disponível em: <<https://keras.io/>>. Acesso em: 4 out. 2021.

³ Disponível em: <<https://github.com/Theano/Theano>>. Acesso em: 4 out. 2021.

- possui alta escalabilidade de computação.

A arquitetura do Keras é composta de três categorias:

- *model* (modelo);
- *layer* (camada); e
- *core modules* (módulos principais).

Um *Model* é composto de vários *Layers*, que podem ser camadas de entrada, ocultas, saída, convolutivas, *pooling* etc. Em *core modules* estão as funções de ativação, datasets, otimizadores, funções de custo, métricas etc., que são utilizadas pelas camadas (Keras, 2019).

Keras permite a definição de um modelo denominado **Sequential** (sequencial). É uma espécie de pilha de camadas que possuem um tensor de entrada e um tensor de saída. As diferentes camadas de uma rede convolucional podem ser definidas, permitindo flexibilidade no desenvolvimento e o uso de uma série de recursos para o refinamento do modelo a ser criado. Na figura 16 pode-se verificar um script no qual há a criação de um modelo (*model*) no qual uma camada totalmente conectada (*Dense*) é adicionada contendo 10 unidades, utilizando a função de ativação sigmoide. Deve-se notar também que na primeira e segunda linhas estão os comandos necessários para importar os objetos *Sequential*, *Dense* e *Activation* da biblioteca Keras.

Figura 16 – Exemplo de um modelo em Keras

```
In [1]: from keras.models import Sequential
        from keras.layers import Dense, Activation

        model = Sequential()
        model.add(Dense(10, activation='sigmoid', input_shape=(128,)))
```

Quadro 1 – Métodos para uso com Sequential

Método	Descrição	Exemplo
add	Adiciona uma camada ao modelo criado com Sequential.	<code>model.add(Conv2D, 32, (3, 3))</code>
compile	Compila o modelo, informando a função custo, algoritmo de treinamento (otimizador) e a métrica a ser utilizada.	<code>model.compile(loss='mean_squared_error', optimizer='sgd', metrics=[metrics.categorical_accuracy])</code>
fit	Treina o modelo, sendo informado diversos parâmetros como conjunto de entrada e saída, tamanho do lote, dados para teste e validação.	<code>model.fit(x_train, y_train, batch_size=128, epochs=20, verbose=1, validation_data=(x_test, y_test))</code>
predict	Faz a predição com o modelo treinado de um conjunto de validação.	<code>predict(x, batch_size=32)</code>
evaluate	Avalia a performance do modelo construído.	<code>model.evaluate(x_test, y_test)</code>

O Quadro 1 mostra uma série de métodos que são utilizados para a construção, treinamento e avaliação do modelo criado com Sequential. Keras também implementa métodos para carregar datasets bastante utilizados em problemas de classificação como o MNIST e o CIFAR-10, por exemplo (figura 17).

Portanto, pode-se constatar que a biblioteca Keras, junto com Tensorflow, constituem um recurso muito intuitivo e flexível, permitindo a construção de modelos de forma bastante interativa e possibilitando um refinamento gradativo para atender a um determinado problema de classificação dentro do contexto de deep learning.

Figura 17 – Métodos para importação de datasets

```
In [2]: from keras.datasets import mnist, cifar10, imdb

(x_train, y_train) = mnist.load_data()
(x_train2, y_train2) = cifar10.load_data()
(x_train3, y_train3) = imdb.load_data()
```

Além de Keras e Tensorflow, outras bibliotecas também estão disponíveis, tais como PyTorch⁴, Chainer⁵ e Apache MXNet Gluon⁶.

TEMA 4 – CONSTRUÇÃO DO MODELO EM PYTHON E JUPYTER NOTEBOOK

Para a compreensão sobre a estrutura e a dinâmica de uma rede convolutiva, será construído um modelo para classificação de um *dataset* popular denominado **Fashion MNIST**⁷ (Xiao; Rasul; Vollgraf, 2017). Este *dataset* compõe-se de um conjunto de treinamento de 60.000 imagens e um conjunto de teste de 10.000 imagens. As imagens têm dimensões 28x28 em escala de cinza, associadas com 10 classes (figura 18).

A ideia é construir um modelo de rede convolucional, com um determinado número de camadas e que possa ser treinado com as 60.000 imagens. As 10.000 imagens são utilizadas para fazer a validação do modelo. Na figura 19 consta a forma da **carga dos dados** dentro do Jupyter Notebook.

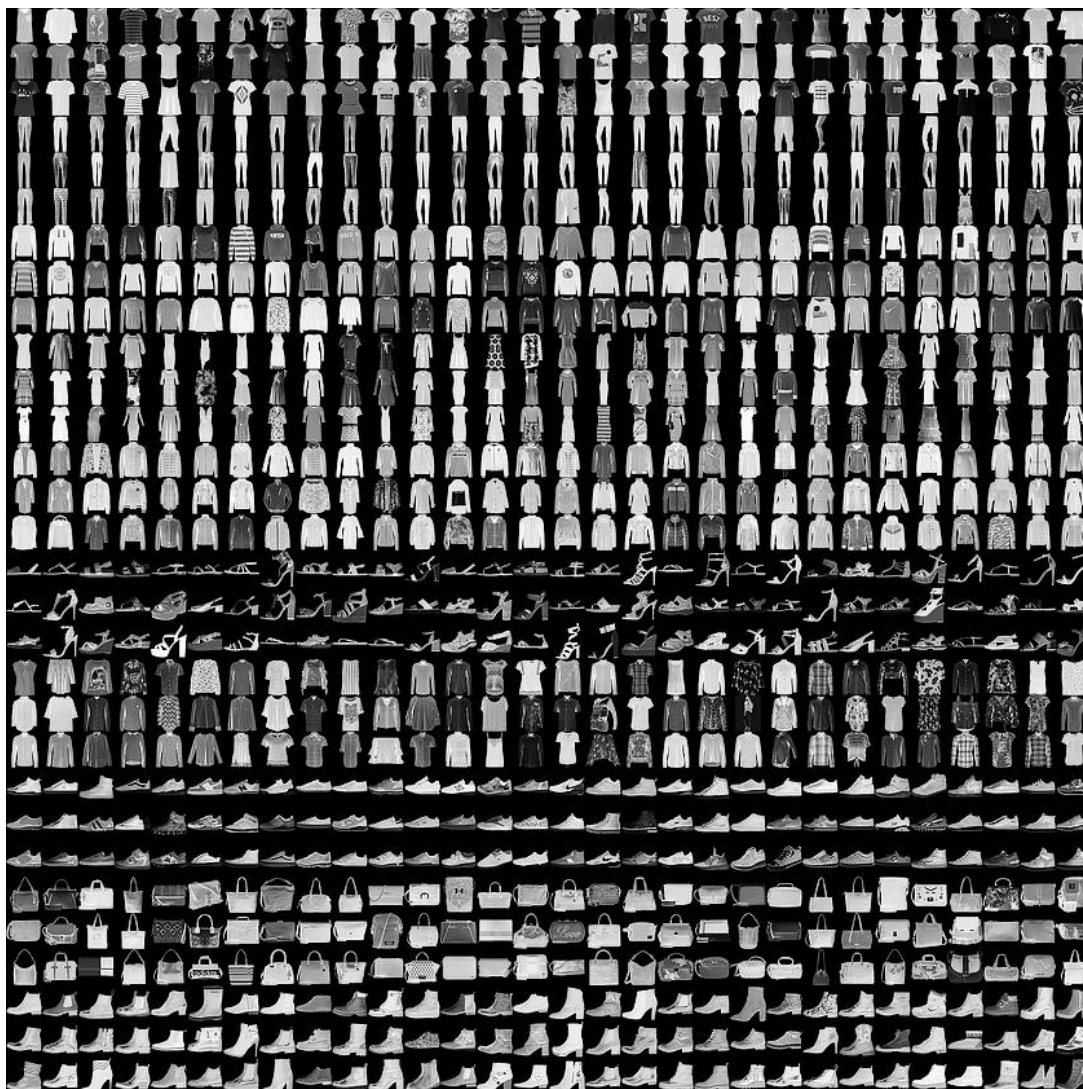
⁴ Disponível em: <<https://pytorch.org/>>. Acesso em: 4 out. 2021.

⁵ Disponível em: <<https://docs.chainer.org/>>. Acesso em: 4 out. 2021.

⁶ Disponível em: <<https://mxnet.apache.org/>>. Acesso em: 4 out. 2021.

⁷ Disponível em: <<https://github.com/zalandoresearch/fashion-mnist#pca-on-fashion-mnist-left-and-original-mnist-right>>. Acesso em: 4 out. 2021.

Figura 18 – Exemplos de imagens do *dataset* Fashion-MNIST. Cada grupo de três linhas corresponde a uma classe de acessórios



Fonte: GitHub, [S.d.].

Figura 19 – Carga do *dataset* Fashion-MNIST

```
In [1]: from keras.datasets import fashion_mnist
        (train_X, train_Y), (test_X, test_Y) = fashion_mnist.load_data()
```

É oportuno proceder a uma **análise do dataset** para saber como os dados estão estruturados e verificar se há a necessidade de alteração em alguma estrutura. O script na Figura 20 confirma o tamanho das imagens 28x28 do conjunto de entrada (60.000) e de validação ou teste (10.000). O script seguinte mostra a quantidade de classes do conjunto de treinamento, com as classes de saída etiquetadas de “0” a “9”.

Figura 20 – Análise das características do *dataset*

```
In [2]: import numpy as np
        from keras.utils import to_categorical
        import matplotlib.pyplot as plt
        %matplotlib inline

        print('Training data shape : ', train_X.shape, train_Y.shape)
        print('Testing data shape : ', test_X.shape, test_Y.shape)

Training data shape : (60000, 28, 28) (60000,)
Testing data shape : (10000, 28, 28) (10000,)

In [3]: # Encontra a quantidade de classes do conjunto de treinamento
        classes = np.unique(train_Y)
        nClasses = len(classes)
        print('Total number of outputs : ', nClasses)
        print('Output classes : ', classes)

Total number of outputs : 10
Output classes : [0 1 2 3 4 5 6 7 8 9]
```

Para a verificação das primeiras imagens do *dataset*, utilizam-se as APIs da Matplotlib para mostrar as primeiras imagens do conjunto de treinamento e do conjunto de teste. Na Figura 21 estão descritos os scripts. O método `subplot` posiciona na área de impressão; o método `imshow` imprime a imagem 28x28; e `title` é usado para mostrar a classe a que pertence a imagem.

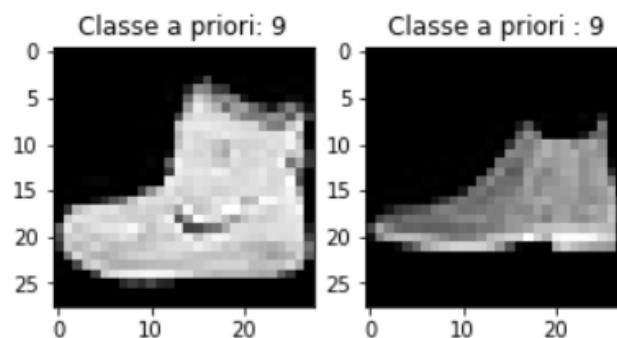
Figura 21 – Verificação das primeiras imagens dos conjuntos, treinamento e teste

```
In [4]: plt.figure(figsize=[5,5])

# Mostra a primeira imagem do conjunto de treinamento
plt.subplot(121)
plt.imshow(train_X[0,:,:], cmap='gray')
plt.title("Classe a priori: {}".format(train_Y[0]))

# Mostra a primeira imagem do conjunto de teste
plt.subplot(122)
plt.imshow(test_X[0,:,:], cmap='gray')
plt.title("Classe a priori : {}".format(test_Y[0]))
```

Out[4]: Text(0.5, 1.0, 'Classe a priori : 9')



É necessário que se faça a conversão das estruturas das imagens em tensores para que possa constituir a primeira camada do modelo a ser criado. Como as imagens têm tamanho 28x28, é necessário reescalar para a faixa [0,1] e reformatar para que seja um tensor de tamanho 28x28x1, para que possa servir como camada de entrada para a rede convolucional. Os scripts da figura 20 fazem esta formatação, mudam os valores dos pixels das imagens da escala de cinza para a faixa [0,1] e criam uma representação vetorial para as classes de cada imagem.

Em seguida, faz-se a **preparação dos dados** ou **pré-processamento**. Como as imagens estão em tons de cinza, os valores dos pixels (em formato de inteiro de 8 bits) estão dispostos no intervalo [0,255]. Portanto, é necessário fazer a transformação dos valores em ponto flutuante para o processamento da rede convolucional. Na Figura 22 constam os scripts para estas operações.

Figura 22 – Scripts para a modificação das imagens para tensores, ajuste da escala de valores dos pixels das imagens e a mudança da representação da classe em formato vetorial

```
In [5]: # Formata o conjunto das imagens (treinamento e teste) para um tensor 28x28x1
train_X = train_X.reshape(-1, 28,28, 1)
test_X = test_X.reshape(-1, 28,28, 1)
train_X.shape, test_X.shape

Out[5]: ((60000, 28, 28, 1), (10000, 28, 28, 1))

In [6]: # Ajuste da escala dos valores para a faixa [0,1] e mudança do tipo para ponto flutuante de 32 bits.
train_X = train_X.astype('float32')
test_X = test_X.astype('float32')
train_X = train_X / 255.
test_X = test_X / 255.

In [7]: # Modifica a representação das classes de categórica para vetorial
train_Y_flat = to_categorical(train_Y)
test_Y_flat = to_categorical(test_Y)

# Mostra a modificação
print('Classe original:', train_Y[0])
print('Classe em formato vetorial:', train_Y_flat[0])

Classe original: 9
Classe em formato vetorial: [0. 0. 0. 0. 0. 0. 0. 0. 0. 1.]
```

A partir daqui, faz-se a preparação do conjunto de treinamento. O treinamento considera que o conjunto de 60.000 imagens será dividido de forma que 80% fará parte do conjunto de treinamento propriamente dito (ou seja, 48.000 imagens) e 20% (12.000 imagens) será utilizado para a validação após cada lote durante o treinamento (figura 23). Não deve ser confundido este conjunto de teste com o outro conjunto das 10.000 imagens que serão utilizadas para a avaliação do modelo construído. Utiliza-se a função `train_test_split`, que divide o conjunto e prepara para o treinamento, de acordo com o valor de `test_size`. O uso posterior do método `shape` mostra o tamanho dos conjuntos a serem utilizados.

Figura 23 – Divisão do conjunto de treinamento

```
In [8]: from sklearn.model_selection import train_test_split
train_X,valid_X,train_label,valid_label = train_test_split(train_X, train_Y_flat, test_size=0.2, random_state=13)

In [9]: train_X.shape,valid_X.shape,train_label.shape,valid_label.shape

Out[9]: ((48000, 28, 28, 1), (12000, 28, 28, 1), (48000, 10), (12000, 10))
```

Com estas operações prévias executadas, parte-se para a **construção do modelo** propriamente dita. É necessário fazer a importação da API do Keras com os módulos necessários. Na figura 24 está o script que executa esta importação.

Figura 24 – Importação das APIs e definição dos parâmetros de tamanho do lote, número de épocas de treinamento e número de classes

```
In [10]: import keras
         from keras.models import Sequential, Input, Model
         from keras.layers import Dense, Dropout, Flatten
         from keras.layers import Conv2D, MaxPooling2D
         from keras.layers.normalization import BatchNormalization
         from keras.layers.advanced_activations import LeakyReLU

In [11]: batch_size = 64
         epochs = 20
         num_classes = 10
```

A chamada ao objeto `Sequential()` é sucedida pela adição das camadas que farão parte da rede convolucional. Elas serão empilhadas da seguinte forma:

- Convolução, 32 filtros 3x3 com ativação ReLU;
- Max Pooling 2x2;
- Convolução, 64 filtros 3x3 com ativação ReLU;
- Max Pooling 2x2;
- Flatten;
- Dense, 128 unidades; e
- Somam, 10 classes.

As camadas de convolução utilizam um kernel 3x3, gerando 32 imagens 28x28 na primeira convolução. Com *max pooling* de 2x2, na segunda convolução o número de imagens aumenta para 64 de tamanho 14x14. Novamente com *max pooling* 2x2, o tamanho das 64 imagens chegará a 7x7, contendo um total de 3136 unidades (7x7x64). Após a camada *flatten*, para a rede totalmente conectada serão 3136 entradas, com mais uma camada densa (oculta) de 128, totalmente conectada a uma camada de saída com 10 classes, usando ativação *softmax*. Para a função de ativação ReLU após cada convolução, será utilizada a *leaky ReLU* com valor mínimo de 0.1. O número total de parâmetros livres a serem modificados pelo treinamento será a soma de todas as camadas de convolução e densas, totalizando 421.642. Na figura 25 está o script que executa a construção do modelo. Na figura 26 é mostrada a execução do comando

`summary`, que mostra um resumo da criação e detalha a quantidade de parâmetros livres da rede convolucional que foi criada.

Figura 25 – Script para construção do modelo e empilhamento das camadas

```
In [12]: model = Sequential()
model.add(Conv2D(32, kernel_size=(3, 3), activation=None, input_shape=(28,28,1), padding='same'))
model.add(LeakyReLU(alpha=0.1))
model.add(MaxPooling2D((2, 2), padding='same'))
model.add(Conv2D(64, (3, 3), activation=None, padding='same'))
model.add(LeakyReLU(alpha=0.1))
model.add(MaxPooling2D(pool_size=(2, 2), padding='same'))
model.add(Flatten())
model.add(Dense(128, activation='sigmoid'))
model.add(Dense(num_classes, activation='softmax'))
```

Figura 26 – Sumário com o modelo criado, mostrando a estrutura de cada camada e o número de parâmetros livres, ou seja, que serão alterados ao longo do treinamento

```
In [13]: model.summary()

Model: "sequential"

```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 28, 28, 32)	320
leaky_re_lu (LeakyReLU)	(None, 28, 28, 32)	0
max_pooling2d (MaxPooling2D)	(None, 14, 14, 32)	0
conv2d_1 (Conv2D)	(None, 14, 14, 64)	18496
leaky_re_lu_1 (LeakyReLU)	(None, 14, 14, 64)	0
max_pooling2d_1 (MaxPooling2	(None, 7, 7, 64)	0
flatten (Flatten)	(None, 3136)	0
dense (Dense)	(None, 128)	401536
dense_1 (Dense)	(None, 10)	1290

```

Total params: 421,642
Trainable params: 421,642
Non-trainable params: 0
```

TEMA 5 – PRÁTICA DO MODELO

O treinamento do modelo de CNN criada é feito a partir da execução do método `fit`. Na figura 27 está o script simples de uma linha que faz o processo. Note que é criada uma variável (*train*), que conterá o resultado do treinamento. A execução do treinamento será feita em 20 épocas, sendo que a cada época são mostrados o erro e a acurácia interna (do próprio conjunto de treinamento). Como o treinamento é feito em lote, as 48.000 imagens divididas por 64 imagens por lote resultam em 750 lotes para serem processados no treinamento.

Na figura 28 está mostrada a saída da execução deste script, mostrando em cada linha uma época de treinamento diferente. Pode-se verificar que a cada execução de uma época é mostrado o tempo de execução e o valor do erro (*loss*) diminui gradativamente e o valor da acurácia (*accuracy*) cresce gradativamente. Estes indicadores se referem às 48.000 imagens que são usadas efetivamente para treinamento. As variáveis *val_loss* e *val_accuracy* se referem ao erro e à acurácia do conjunto de 12.000 amostras.

Figura 27 – Script para o treinamento da rede convolucional

```
In [15]: train = model.fit(train_X, train_label, batch_size=batch_size, epochs=epochs, verbose=1, validation_data=(valid_X, valid_label))
```

Figura 28 – Execução das 20 épocas do treinamento

```
Epoch 1/20
750/750 [=====] - 22s 29ms/step - loss: 0.7115 - accuracy: 0.7492 - val_loss: 0.3643 - val_accuracy:
0.8784
Epoch 2/20
750/750 [=====] - 22s 29ms/step - loss: 0.3302 - accuracy: 0.8802 - val_loss: 0.3190 - val_accuracy:
0.8825
Epoch 3/20
750/750 [=====] - 22s 29ms/step - loss: 0.2817 - accuracy: 0.8982 - val_loss: 0.2767 - val_accuracy:
0.8991
Epoch 4/20
750/750 [=====] - 22s 29ms/step - loss: 0.2358 - accuracy: 0.9132 - val_loss: 0.2648 - val_accuracy:
0.9038
Epoch 5/20
750/750 [=====] - 21s 29ms/step - loss: 0.2124 - accuracy: 0.9234 - val_loss: 0.2483 - val_accuracy:
0.9071
Epoch 6/20
750/750 [=====] - 21s 28ms/step - loss: 0.1908 - accuracy: 0.9298 - val_loss: 0.2444 - val_accuracy:
0.9087
Epoch 7/20
750/750 [=====] - 21s 28ms/step - loss: 0.1720 - accuracy: 0.9371 - val_loss: 0.2437 - val_accuracy:
0.9129
Epoch 8/20
750/750 [=====] - 22s 29ms/step - loss: 0.1495 - accuracy: 0.9455 - val_loss: 0.2357 - val_accuracy:
0.9153
Epoch 9/20
750/750 [=====] - 21s 28ms/step - loss: 0.1332 - accuracy: 0.9525 - val_loss: 0.2320 - val_accuracy:
0.9158
Epoch 10/20
750/750 [=====] - 23s 30ms/step - loss: 0.1144 - accuracy: 0.9587 - val_loss: 0.2382 - val_accuracy:
0.9172
Epoch 11/20
750/750 [=====] - 23s 30ms/step - loss: 0.1017 - accuracy: 0.9648 - val_loss: 0.2275 - val_accuracy:
0.9236
Epoch 12/20
750/750 [=====] - 23s 31ms/step - loss: 0.0849 - accuracy: 0.9713 - val_loss: 0.2559 - val_accuracy:
0.9172
Epoch 13/20
750/750 [=====] - 22s 30ms/step - loss: 0.0767 - accuracy: 0.9752 - val_loss: 0.2326 - val_accuracy:
0.9226
Epoch 14/20
750/750 [=====] - 22s 29ms/step - loss: 0.0616 - accuracy: 0.9806 - val_loss: 0.2444 - val_accuracy:
0.9219
Epoch 15/20
750/750 [=====] - 23s 30ms/step - loss: 0.0542 - accuracy: 0.9839 - val_loss: 0.2494 - val_accuracy:
0.9178
Epoch 16/20
750/750 [=====] - 23s 30ms/step - loss: 0.0434 - accuracy: 0.9878 - val_loss: 0.2516 - val_accuracy:
0.9209
Epoch 17/20
750/750 [=====] - 23s 30ms/step - loss: 0.0352 - accuracy: 0.9916 - val_loss: 0.2596 - val_accuracy:
0.9215
Epoch 18/20
750/750 [=====] - 22s 30ms/step - loss: 0.0300 - accuracy: 0.9922 - val_loss: 0.2781 - val_accuracy:
0.9192
Epoch 19/20
750/750 [=====] - 23s 31ms/step - loss: 0.0274 - accuracy: 0.9935 - val_loss: 0.2730 - val_accuracy:
0.9219
Epoch 20/20
750/750 [=====] - 23s 30ms/step - loss: 0.0218 - accuracy: 0.9951 - val_loss: 0.2907 - val_accuracy:
0.9208
```

Para a validação do modelo, utiliza-se o conjunto de teste de 10.000 amostras que não fizeram parte do treinamento. O erro e a acurácia podem ser mostrados também na forma de um histórico, sendo possível verificar como o treinamento foi modificando estes indicadores. Com alguns comandos da API Matplotlib, pode-se mostrar graficamente esta evolução. Na figura 29 constam os scripts para a validação e para a geração dos gráficos que estão mostrados na figura 30.

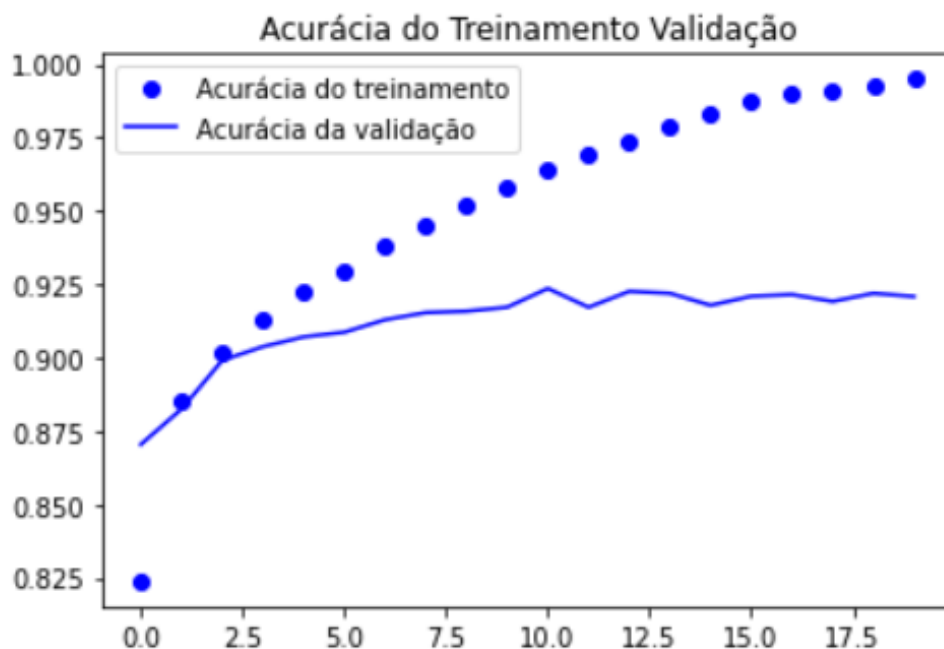
Figura 29 – Scripts para validação e geração do gráfico com a evolução do treinamento

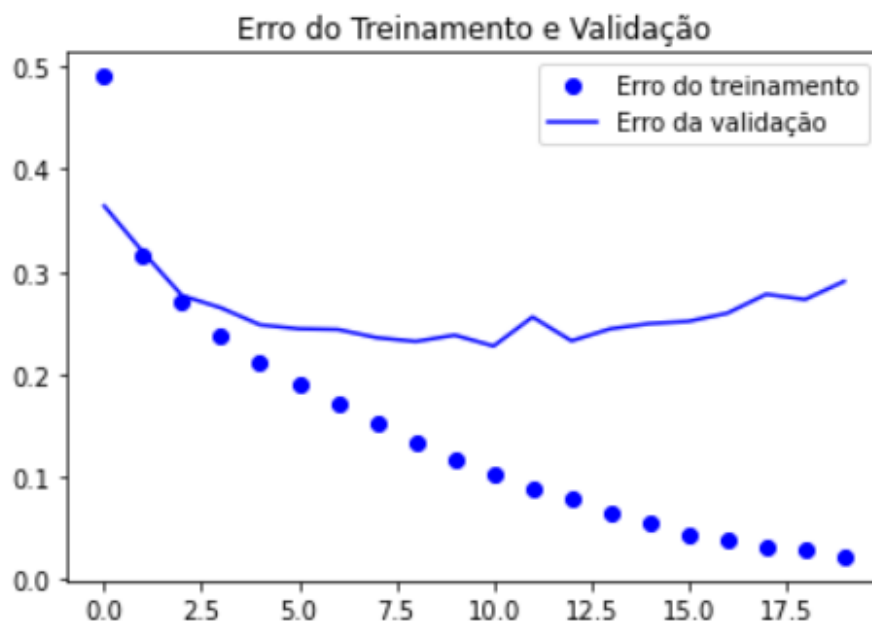
```
In [17]: test_eval = model.evaluate(test_X, test_Y_flat, verbose=1)
313/313 [=====] - 1s 4ms/step - loss: 0.3005 - accuracy: 0.9219

In [18]: print('Test loss:', test_eval[0])
print('Test accuracy:', test_eval[1])
Test loss: 0.30048075318336487
Test accuracy: 0.9218999743461609

In [19]: accuracy = train.history['accuracy']
val_accuracy = train.history['val_accuracy']
loss = train.history['loss']
val_loss = train.history['val_loss']
epochs = range(len(accracy))
plt.plot(epochs, accracy, 'bo', label='Acurácia do treinamento')
plt.plot(epochs, val_accuracy, 'b', label='Acurácia da validação')
plt.title('Acurácia do Treinamento Validação')
plt.legend()
plt.figure()
plt.plot(epochs, loss, 'bo', label='Erro do treinamento')
plt.plot(epochs, val_loss, 'b', label='Erro da validação')
plt.title('Erro do Treinamento e Validação')
plt.legend()
plt.show()
```

Figura 30 – Gráficos mostrando a relação entre a acurácia do treinamento e da validação, bem como do erro





O próximo passo consiste em verificar o nível de predição que o modelo proporciona. A partir daqui, o modelo está pronto para predizer a qual classe as imagens do conjunto sendo testado pertencem. Este passo se encarrega de uma verificação sobre quantas classes são atribuídas corretamente e incorretamente. Algumas imagens específicas são mostradas para indicar estas atribuições. Na figura 31 está o script para a execução da predição, sendo mostradas na sequência as classes que são classificadas corretamente e incorretamente.

Figura 31 – Etapa de predição, com a comparação das classes corretamente e incorretamente atribuídas

```
In [20]: predicted_classes = model.predict(test_X)

In [21]: predicted_classes = np.argmax(np.round(predicted_classes),axis=1)

In [22]: predicted_classes.shape, test_Y.shape
Out[22]: ((10000,), (10000,))

In [23]: correct = np.where(predicted_classes==test_Y)[0]
print("Encontrados %d classes corretas" % len(correct))
for i, correct in enumerate(correct[:9]):
    plt.subplot(3,3,i+1)
    plt.imshow(test_X[correct].reshape(28,28), cmap='gray', interpolation='none')
    plt.title("Predito {}, Classe {}".format(predicted_classes[correct], test_Y[correct]))
    plt.tight_layout()

Encontrados 9209 classes corretas
```

```
In [24]: incorrect = np.where(predicted_classes!=test_Y)[0]
print("Encontrados %d classes incorretas" % len(incorrect))
for i, incorrect in enumerate(incorrect[:9]):
    plt.subplot(3,3,i+1)
    plt.imshow(test_X[incorrect].reshape(28,28), cmap='gray', interpolation='none')
    plt.title("Predito {}, Classe {}".format(predicted_classes[incorrect], test_Y[incorrect]))
plt.tight_layout()
```

Encontrados 791 classes incorretas

Por fim, a avaliação do modelo pode ser feita calculando-se os indicadores da matriz de confusão para cada uma das 10 classes. Isto permite calcular a precisão, o recall, o F1-score para cada classe e de todo o modelo, além da acurácia mostrada anteriormente. Na figura 32 pode-se verificar o script que executa esta avaliação, mostrando os indicadores respectivos. Pode-se notar que algumas classes possuem um alto valor em relação a outras.

Figura 32 – Cálculo de indicadores da matriz de confusão para cada classe e no geral do modelo criado

```
In [25]: from sklearn.metrics import classification_report
target_names = ["Classe {}".format(i) for i in range(num_classes)]
print(classification_report(test_Y, predicted_classes, target_names=target_names))
```

	precision	recall	f1-score	support
Classe 0	0.81	0.92	0.86	1000
Classe 1	0.98	0.99	0.99	1000
Classe 2	0.87	0.88	0.87	1000
Classe 3	0.93	0.90	0.92	1000
Classe 4	0.86	0.89	0.88	1000
Classe 5	0.99	0.98	0.99	1000
Classe 6	0.85	0.72	0.78	1000
Classe 7	0.96	0.99	0.97	1000
Classe 8	0.99	0.98	0.98	1000
Classe 9	0.98	0.96	0.97	1000
accuracy			0.92	10000
macro avg	0.92	0.92	0.92	10000
weighted avg	0.92	0.92	0.92	10000

De forma geral, o treinamento de uma rede convolucional pode ser refinado a partir da modificação dos parâmetros (tamanho de lote, número de épocas, divisão do conjunto de treinamento) ou mesmo da construção do modelo, por meio da introdução de mais camadas ou modificação das existentes, modificação das funções de ativação, tamanho dos kernels das convoluções etc. Por tentativa e erro, modelos com mais acurácia podem ser explorados.

REFERÊNCIAS

BATTITI, R.; BRUNATO, M. **The LION Way: Machine Learning plus Intelligent Optimization**. Trento: University of Trento, 2017. E-book.

BENGIO, Y. et al. Greedy Layer-Wise Training of Deep Networks. **Advances in neural information processing systems**, [S.l.], v. 19, n. 1, p. 153, 2007.

CONVNETJS CIFAR-10 DEMO. Stanford University, [S.d.]. Disponível em:

<<https://cs.stanford.edu/people/karpathy/convnetjs/demo/cifar10.html>>. Acesso em: 4 out. 2021.

FASHION-MNIST. GitHub, [S.d.]. Disponível em:

<<https://github.com/zalandoresearch/fashion-mnist#pca-on-fashion-mnist-left-and-original-mnist-right>>. Acesso em: 4 out. 2021.

GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. **Deep Learning**. Cambridge-MA: MIT Press, 2016. E-book.

HINTON, G. E.; OSINDERO, S.; TEH, Y.-W. A fast learning algorithm for deep belief nets. **Neural Comput.**, Cambridge, MA, USA, v. 18, n. 7, p. 1527–1554, 2006. Doi: <<https://doi.org/10.1162/neco.2006.18.7.1527>>.

HOCHREITER, S.; SCHMIDHUBER, J. Long Short-Term Memory. **Neural Computation**, [S.l.], v. 9, n. 8, p. 1735–1780, 1997. Doi: <<https://doi.org/10.1162/neco.1997.9.8.1735>>.

KERAS. [S.l.], 2019. Available at: <https://www.tutorialspoint.com/keras/keras_tutorial.pdf>. Acesso em: 4 out. 2021.

KETKAR, N. **Deep Learning with Python: a hands-on introduction**. California: Apress Media, 2017.

KRIZHEVSKY, A.; SUTSKEVER, I.; HINTON, G. E. ImageNet Classification with Deep Convolutional Neural Networks. In: **NIPS 2012: Neural Information Processing Systems**, Lake Tahoe, Nevada: [S.n.], 2012. Available at: <http://www.cs.toronto.edu/~kriz/imagenet_classification_with_deep_convolutional.pdf>. Acesso em: 4 out. 2021.

LECUN, Y. et al. Gradient-based learning applied to document recognition. **Proceedings of the IEEE**, [S.l.], v. 86, n. 11, p. 2278–2324, 1998. Doi: <<https://doi.org/10.1109/5.726791>>.

MEDEIROS, L. F. de. **Inteligência Artificial Aplicada: uma abordagem introdutória**. Curitiba: InterSaberes, 2018.

NIELSEN, M. **Neural Networks and Deep Learning**. [S.l.: S.n.], 2018. E-book.

RANZATO, M. A. et al. Efficient learning of sparse representations with an energy-based model. **Advances in Neural Information Processing Systems**, [S.l.], p. 1137–1144, 2007. Doi: <<https://doi.org/10.7551/mitpress/7503.003.0147>>.

XIAO, H.; RASUL, K.; VOLLGRAF, R. **Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms**. [S.l.: S.n.], 2017.